Planning and Self-Reflection in Large Language Model Reasoning

Abstract

The integration of planning and self-reflection mechanisms has emerged as a pivotal paradigm for enhancing the reasoning capabilities of Large Language Models (LLMs). Moving beyond static, single-pass generation, contemporary approaches treat reasoning as a dynamic, iterative process involving explicit plan formulation, execution monitoring, error detection, and corrective refinement. This survey provides a comprehensive synthesis of 300 recent academic contributions, categorizing methodological advances into distinct families: anticipatory planning architectures, iterative self-correction loops, search-based reasoning frameworks, representation-level interventions, and training paradigms for internalizing reflective behaviors. We analyze how methods such as Tree of Thoughts, Plan of Thoughts, and various self-reflection loops leverage LLMs as both generators and verifiers to navigate complex solution spaces. Furthermore, we examine the theoretical underpinnings of these systems, including their formulation as Markov Decision Processes and their reliance on latent world models. Despite significant progress, challenges remain regarding the reliability of intrinsic verification, computational efficiency, and the generalization of reflective strategies across domains. This paper critically evaluates the state-of-the-art, highlighting the trade-offs between inference-time compute and reasoning accuracy, and outlines future directions for robust, autonomous reasoning agents.

1. Introduction

The evolution of Large Language Models (LLMs) from static text predictors to dynamic reasoning engines marks a fundamental shift in artificial intelligence. Early successes relied on scaling laws and next-token prediction, yet these models often struggled with multi-step problems requiring long-horizon dependency tracking and logical consistency [1]. The introduction of Chain-of-Thought (CoT) prompting demonstrated that eliciting intermediate reasoning steps could significantly improve performance on arithmetic and logical tasks [2]. However, standard CoT remains largely linear and prone to error propagation; a single mistake in an early step often derails the entire trajectory without opportunity for recovery [3].

To address these limitations, the field has rapidly converged on two complementary mechanisms: **planning** and **self-reflection**. Planning involves the explicit decomposition of complex goals into manageable sub-tasks or the exploration of multiple potential solution paths before commitment [4]. Self-reflection introduces a meta-cognitive layer where the model critiques its own outputs, identifies inconsistencies, and iteratively refines its reasoning [5]. Together, these mechanisms transform LLM inference from a feed-forward process into a closed-loop control system capable of backtracking, verification, and adaptive strategy selection.

Recent literature reveals a diverse ecosystem of approaches. Some methods externalize planning through search algorithms like Monte Carlo Tree Search (MCTS), treating the LLM as a heuristic function within a broader optimization framework [6]. Others internalize these capabilities via fine-tuning, teaching models to generate "thought tokens" or engage in multi-turn self-dialogue [7]. A critical distinction has emerged between *intrinsic* self-correction, where the model relies solely on its parametric knowledge to detect errors, and *extrinsic* correction, which utilizes external tools, formal verifiers, or multi-agent debate [3]. While intrinsic methods offer efficiency, they often suffer from hallucinated constraints and false positives [3]. Conversely, extrinsic methods provide soundness guarantees but at increased computational cost and latency [8].

This survey synthesizes findings from 300 relevant papers to provide a structured overview of planning and self-reflection in LLM reasoning. We organize the field into coherent methodological families, analyzing the technical innovations, empirical results, and inherent limitations of each. We begin by examining anticipatory planning and abstract meta-knowledge generation, followed by a deep dive into iterative self-correction frameworks. We then explore search-based reasoning systems that formalize problem-solving as Markov Decision Processes (MDPs) or Partially Observable MDPs (POMDPs). Subsequently, we discuss advances in representation engineering and inference-time interventions that steer model activations toward reflective states. Finally, we review training paradigms designed to internalize these behaviors and conclude with a discussion of open challenges, including verification reliability, efficiency bottlenecks, and safety concerns.

2. Method

The methodological landscape of planning and self-reflection in LLMs can be broadly categorized into five interconnected dimensions: (1) Anticipatory Planning and Abstract Meta-Knowledge, (2) Iterative Self-Correction and Reflection Loops, (3) Search-Based and Tree-Structured Reasoning, (4) Representation Engineering and Inference-Time Intervention, and (5) Training Paradigms for Internalized Reflection. Each dimension addresses specific failure modes of standard autoregressive generation, ranging from myopic decision-making to uncorrected error propagation.

2.1 Anticipatory Planning and Abstract Meta-Knowledge

A primary limitation of standard CoT is its reactive nature; models often begin solving before fully understanding the problem structure, leading to cognitive overload and premature convergence on suboptimal paths. Anticipatory planning methods address this by decoupling the formulation of a high-level strategy from the detailed execution.

Abstract Plan Generation. The LEarning to Plan before Answering (LEPA) framework exemplifies this approach by iteratively training models to generate abstract anticipatory plans prior to solution derivation [9]. LEPA distinguishes between problem-specific details and transferable meta-knowledge, ensuring that plans remain generalizable. By formulating these plans before engaging in detailed solving, the method reduces cognitive load and distills reusable strategies. Evaluation on the Hendrycks MATH dataset shows that LEPA achieves a test accuracy of 30.2%, outperforming baselines like ReST and STaR, although it initially lags on benchmarks like Hellaswag until sufficient self-training occurs [9]. Similarly, the CODEPLAN method frames high-level planning as code generation, utilizing Python-style pseudocode to capture control flows [10]. This structured representation leverages the semantic versatility of code to enable better generalization than natural language plans, yielding a 25.1% relative improvement across 13 benchmarks [10].

Hierarchical and Decomposed Planning. Complex tasks often require multi-level decomposition. HyperTree Planning (HTP) constructs hypertree-structured outlines where parent nodes connect to sets of child nodes, facilitating a top-down divide-and-conquer strategy [11]. This approach allows for flexible hierarchical thinking, achieving a 3.6x improvement in success rate on the TravelPlanner dataset compared to o1-preview [11]. In the domain of embodied AI, EvolveNav employs a two-stage protocol where formalized CoT labels predict specific landmarks and directions, streamlining reasoning generation for vision-language navigation [12]. This formalization reduces redundant tokens and improves inference speed while maintaining high performance on the R2R dataset [12].

Intent and Constraint Modeling. Explicitly modeling intent and constraints further refines

planning. The Speaking with Intent (SWI) method injects free-form intent statements into the generation stream, directing autoregressive generation toward high-level goals before executing specific steps [13]. This synergizes with CoT to improve factual consistency in summarization and mathematical reasoning [13]. Conversely, Constraints-of-Thought (Const-o-T) decomposes reasoning steps into `<intent, constraint>` pairs, where constraints serve as machine-executable symbolic instructions that actively prune infeasible branches during Monte Carlo Tree Search (MCTS) [14]. This dual-representation transforms rationales into actionable controllers, surpassing vanilla MCTS on risk assessment tasks [14].

Backward and Bidirectional Planning. While most models excel at forward generation, many planning problems benefit from backward reasoning. Fwd-Flip addresses this asymmetry by transforming the original problem—swapping initial and goal states—to create a "flipped" instance solvable via forward generation [15]. By sampling candidate plans for both original and flipped problems and combining them, Fwd-Flip achieves a 24% improvement in success rate on graph planning tasks [15]. This highlights that leveraging forward generation capabilities for backward reasoning tasks can effectively mitigate planning asymmetry.

2.2 Iterative Self-Correction and Reflection Loops

Self-reflection mechanisms enable models to critique and refine their outputs iteratively. These methods range from simple prompt-based loops to complex multi-agent architectures and dynamic instruction systems.

Dynamic Instruction and Meta-Reasoning. The Instruct-of-Reflection (IoRT) method employs a three-stage dynamic loop involving a meta-thinker, a reflector, and an instructor [5]. The meta-thinker retrieves abstract principles, the reflector produces candidates, and the instructor evaluates consistency to issue "refresh," "stop," or "select" commands. This adaptive instruction generation stabilizes reflection trajectories, yielding an average accuracy improvement of 10.1% on datasets like GSM8K and StrategyQA [5]. Similarly, Multi-Layered Self-Reflection with Auto-Prompting (MAPS) uses meta-prompting to generate tailored self-reflection prompts specific to the detected error pattern, significantly reducing symbolic loss compared to fixed templates [16].

Multi-Perspective and Dialectical Reflection. Unidimensional introspection often fails to uncover diverse reasoning flaws. MyGO Poly-Reflective Chain-of-Thought (PR-CoT) executes a three-stage pipeline where parallel reflections target logical consistency, information completeness, ethics, and alternatives [17]. This structured multi-angle critique uncovers a broader spectrum of errors, improving error correction rates by 6% over standard multi-perspective CoT on ethical decision-making tasks [17]. Taking a philosophical approach, the Hegelian Self-Reflection framework executes iterative cycles of proposition, opposition (sublation), and unification (speculation) [18]. By generating opposing critiques and dynamically annealing temperature to balance creativity and convergence, this method forces the model to address inherent flaws, preventing premature convergence [18].

Confidence-Triggered and Online Correction. Efficient reflection requires knowing *when* to correct. Reflective Confidence (ReflectiveConf) computes token-level confidence signals to trigger reflection only when uncertainty exceeds an adaptive threshold [19]. This transforms low-confidence termination signals into active self-correction triggers, salvaging deviating paths more effectively than naive retry mechanisms and achieving 83.3% accuracy on AIME 2025 [19]. At the architectural level, Self-Reflective Generation at Test Time (SRGen) embeds a dynamic uncertainty monitor that pauses decoding upon detecting entropy spikes to optimize a transient correction vector [20]. While

this proactive intervention prevents error cascades, it increases inference latency by approximately 50% due to on-the-fly gradient optimization [20].

Limitations of Intrinsic Verification. Despite these advances, purely intrinsic self-verification faces fundamental limits. Studies show that LLMs acting as verifiers exhibit high false negative rates, often rejecting valid solutions and degrading performance via compounding errors [3]. On the Game of 24 task, self-critique loops performed worse than standard prompting due to hallucinated constraints and incorrect state tracking [3]. Furthermore, synthetic error injection during training often fails to generalize to on-policy errors, as models learn to correct artificial perturbations rather than developing robust internal error detection [21]. Consequently, many high-performing systems now integrate external verifiers or sound symbolic tools to ground the reflection process [3].

2.3 Search-Based and Tree-Structured Reasoning

Formalizing reasoning as a search problem allows LLMs to explore multiple paths, backtrack from dead ends, and utilize heuristics for value estimation. This section covers methods that integrate classical search algorithms with LLM generation.

Tree of Thoughts and Variants. The Tree of Thoughts (ToT) framework decomposes problems into coherent thought steps forming a search tree, employing BFS or DFS to explore diverse candidates [22]. ToT enables deliberate "System 2" processing, achieving a 74% success rate on the Game of 24 task, a significant improvement over the 4% achieved by GPT-4 CoT [22]. Enhancements to ToT include Novelty-based Tree-of-Thought (Novelty-ToT), which integrates width-based planning concepts to prune redundant states via binary LLM-driven novelty estimation [23]. This approach maintains accuracy while reducing token usage by 20x on Blocksworld tasks [23]. Another variant, Comparison-based Tree-of-Thoughts (C-ToT), replaces noisy pointwise scoring with iterative pairwise comparisons, leveraging the LLM's superior relative judgment capabilities to select promising intermediate thoughts [24].

Monte Carlo Tree Search (MCTS) Integration. MCTS provides a robust framework for balancing exploration and exploitation. Reasoning via Planning (RAP) repurposes a frozen LLM as both a reasoning agent and a world model to predict state transitions within an MCTS framework [6]. By explicitly modeling world states, RAP allows the LLM to backtrack from dead ends, surpassing CoT by 33% on Blocksworld tasks [6]. Similarly, Language Agent Tree Search (LATS) adapts MCTS to language agents by defining nodes as states containing action sequences and environmental observations, enabling reverting to previous states via context manipulation [8]. LATS achieves state-of-the-art performance on HumanEval (92.7% Pass@1) by integrating verbal self-reflection into the search process [8].

Heuristic-Guided and POMDP Formulations. Advanced formulations treat reasoning as a Partially Observable Markov Decision Process (POMDP). Plan of Thoughts (PoT) defines states as problem sub-steps and actions as continue/rollback/think, employing a modified POMCP solver with LLM-generated value judgments [4]. This online solver enables anytime performance, dynamically adjusting computation based on problem difficulty [4]. In the domain of heuristic optimization, Planning of Heuristics (PoH) formulates heuristic refinement as an MDP where LLM self-reflection generates actions based on performance gaps, effectively navigating complex heuristic spaces for combinatorial optimization [25].

Graph and Network Structures. Beyond trees, graph structures allow for more complex dependencies. The Knowledgeable Network of Thoughts (kNoT) enables LLMs to autonomously generate

executable workflow scripts, creating a network of thoughts with indexing capabilities [26]. This reduces human labor and allows dynamic adaptation of reasoning structures, outperforming ToT on sorting tasks [26]. Similarly, Structure-aware Planning with an Accurate World Model (SWAP) formulates reasoning as an MDP over entailment graphs, using diversity-based modeling to sample actions and contrastive ranking to evaluate them [27]. This explicit graph structure prevents error propagation by enabling symbolic verification of intermediate steps [27].

2.4 Representation Engineering and Inference-Time Intervention

Recent work explores manipulating the internal latent representations of LLMs to induce planning and reflection behaviors without extensive retraining. These methods leverage the hypothesis that reasoning capabilities are encoded in specific activation patterns.

Activation Steering and Vector Injection. ReflCtrl controls LLM reflection by identifying a "reflection direction" in latent space, computed as the mean difference between reflection and non-reflection step activations [28]. By applying stepwise activation steering at thinking step boundaries, ReflCtrl reduces token usage by 33.6% on GSM8k with minimal accuracy loss, demonstrating that reflection correlates strongly with internal uncertainty [28]. Similarly, Self-Reflection Vector Steering computes a difference-in-means vector between hidden states of reflection-inducing and non-reflection-inducing tokens, injecting it into the residual stream to enhance reasoning performance [29]. This method improved Pass@1 on MATH500 by 12.0% for Qwen2.5-7B while reducing generation length [29].

Error Manifold and Phase-Shift Detection. Errors often manifest as specific geometric deviations in activation space. Manifold-Guided Attention Steering (MAGS) constructs low-dimensional error subspaces via PCA on contrastive traces and steers attention heads away from these manifolds when proximity scores exceed a threshold [30]. This trajectory-aware dynamic steering improved accuracy on MATH-500 by 5.2% [30]. Latent Phase-Shift Rollback (LPSR) monitors residual stream directions to detect abrupt phase shifts indicating reasoning errors, triggering a KV-cache rollback to a previous step [31]. This dual-gate authentication system allows for precise error localization and correction, improving accuracy by 15.2 percentage points over standard autoregressive decoding [31].

Attention Alignment and Anchoring. Structured planning can serve as a scaffold for attention mechanisms. SELF-ANCHOR decomposes reasoning trajectories into explicit plan steps and dynamically selects anchor tokens comprising the original question and current plan step [32]. By applying selective prompt attention to these anchors, the method maintains focus on problem constraints, yielding over 10% accuracy improvement on GSM8K [32]. Thought Anchors attribution analysis further reveals that planning and backtracking sentences act as "thought anchors" with outsized counterfactual impact, attended to by specific "receiver heads" in the transformer architecture [33].

Closed-Loop and Equilibrium Transformers. Architectural modifications can enforce iterative refinement. Equilibrium Transformers (EqT) augment standard layers with an Equilibrium Refinement Module that solves a regularized energy minimization problem in latent space before token commitment [34]. This closed-loop prediction principle approximates MAP inference, correcting representational errors where one-shot proposals fail [34]. Similarly, Attractor Models treat latent refinement as a fixed-point problem, using a smaller attractor module to refine backbone outputs to equilibrium via Anderson acceleration [35]. This enables adaptive computation depth and constant-memory training [35].

2.5 Training Paradigms for Internalized Reflection

While inference-time methods offer flexibility, training paradigms aim to internalize planning and reflection capabilities into the model parameters, enabling faster and more robust reasoning.

Self-Training and Distillation. Self-training frameworks leverage the model's own outputs to generate training data. ROSD (Reflective On-policy Self-Distillation) employs a self-reflector to compare student rollouts against correct solutions, generating corrective ideas that condition a self-teacher [36]. This error-focused distillation restricts gradient updates to verified errors, improving mean accuracy on SciKnowEval [36]. Similarly, Self-Taught Self-Correction (STaSC) iteratively samples answers and corrections, filtering trajectories based on reward improvement to fine-tune the model [37]. This unification of STaR and self-correction algorithms demonstrates that training exclusively on corrections can evolve initialization strategies for better performance [37].

Reinforcement Learning with Verifiable Rewards. Reinforcement Learning (RL) is widely used to optimize reasoning trajectories. ThinkTwice alternates between reasoning optimization and refinement optimization within a unified GRPO framework, creating an emergent curriculum where refinement corrects errors early [38]. This joint training improved pass@4 on AIME by 11.5 points [38]. ScRPO (Self-correction Relative Policy Optimization) focuses training on high-variance negative samples to maximize information gain at the model's knowledge boundary [39]. By attributing rewards exclusively to successful corrections, ScRPO improves accuracy on mathematical benchmarks by 3.2% [39]. Additionally, Cognitive Behavior Priming isolates specific reasoning behaviors like verification and backtracking, priming base models with synthetic datasets before RL training to enable effective test-time compute scaling [40].

Curriculum Learning and Data Synthesis. Effective training often requires curated data. COTEVOL treats reasoning trajectories as individuals in a genetic algorithm, evolving them via reflective global crossover and uncertainty-guided mutation [41]. This reformulation of CoT synthesis enables efficient exploration of reasoning spaces, outperforming standard distillation methods [41]. PLAN-TUNING distills synthetic planning trajectories via Best-of-N sampling, applying two-stage filtering to ensure both plan quality and solution correctness before fine-tuning smaller models [42]. This embedding of explicit natural planning into model parameters yields superior in-domain accuracy and out-of-domain generalization [42].

Multimodal and Domain-Specific Reflection. Reflection training extends to multimodal domains. SRPO (Self-Reflection enhanced reasoning with Group Relative Policy Optimization) implements a two-stage framework for Multimodal LLMs, constructing a reflection-focused SFT dataset where an advanced MLLM generates corrections for policy model outputs [43]. This integration of explicit reflection into both SFT and RL stages surpasses pre-training limits, improving accuracy on MathVista to 78.5% [43]. In medical reasoning, Med-REFL employs Tree-of-Thought to generate diverse trajectories, constructing Direct Preference Optimization pairs to teach models to identify error points by comparing correct and incorrect branches [44].

2.6 Tool Learning and Multi-Agent Collaboration

Planning and reflection are increasingly augmented by external tools and multi-agent interactions, moving beyond the limitations of single-model parametric knowledge.

Tool-Augmented Planning. Methods like ART (Automatic Reasoning and Tool-use) retrieve task-specific program demonstrations to prompt frozen LLMs to generate structured reasoning programs with explicit tool calls [45]. This structured query language approach allows generalization of

decomposition strategies across tasks [45]. Tree-of-Code (ToC) integrates Tree-of-Thought search with CodeAct execution, treating code execution results as tree nodes to systematically explore alternative solutions [46]. This execution-level reflection drives tree expansion, improving accuracy on tool evaluation benchmarks by 7.2% [46]. Furthermore, Tool-MVR integrates Multi-Agent Meta-Verification to construct high-quality tool usage data, validating APIs and trajectories to improve tool planning accuracy by 15.3% over GPT-4 [47].

Multi-Agent Debate and Collaboration. Multi-agent systems leverage diverse perspectives to enhance reflection. RR-MP (Reactive and Reflection agents with Multi-Path Reasoning) deploys parallel reasoning paths where reactive agents generate preliminary answers and reflection agents perform critiques via shared memory [48]. This external stimulation breaks cognitive rigidity, improving MMLU accuracy to 75.94% [48]. Similarly, DuSAR (Dual-Strategy Agent with Reflecting) decouples planning into holistic and local strategies, using a Strategy Fitness Score to trigger dynamic replanning [49]. This co-adaptive dual-strategy framework enables robust zero-shot generalization on ALFWorld tasks [49].

Human-in-the-Loop and Interactive Planning. Human feedback remains crucial for complex planning. Human-Centered Planning (LLMPlan) utilizes a constraint checker to identify violations and verbalize feedback for iterative correction, achieving near-symbolic accuracy on explicit constraints while handling implicit commonsense [50]. In manufacturing, KG-LLM Collaborative Frameworks transform simulation logs into Knowledge Graphs, employing dual-path processing with step-wise Cypher generation and self-reflection to identify operational bottlenecks [51]. This iterative evidence-driven reasoning significantly outperforms monolithic single-pass approaches [51].

3. Challenges and Outlook

Despite the rapid advancement of planning and self-reflection techniques, several critical challenges hinder their widespread deployment and theoretical understanding.

Reliability of Intrinsic Verification. A fundamental bottleneck remains the unreliability of LLMs as intrinsic verifiers. As noted in multiple studies, LLMs often hallucinate constraints or fail to detect subtle logical inconsistencies, leading to false positives and negatives that degrade performance in iterative loops [3, 52]. While methods like Reflect-Guard attempt to distill analytical reasoning into safety classifiers, they still struggle with adversarial framing and over-triggering on benign prompts [53]. Future research must focus on hybrid verification systems that combine intrinsic LLM critique with sound external solvers or formal methods to ensure correctness guarantees, particularly in safety-critical domains [3, 54].

Computational Efficiency and Overthinking. The computational cost of planning and reflection is prohibitive for many applications. Search-based methods like ToT and MCTS require multiple LLM calls per step, leading to exponential increases in latency and token usage [4, 22]. Furthermore, Large Reasoning Models (LRMs) exhibit "overthinking," generating unnecessary reasoning chains for simple queries, which reduces efficiency without improving accuracy [55, 56]. Methods like Think-at-Hard and Confidence-Aware Decision Frameworks attempt to selectively trigger reflection only for hard tokens or uncertain predictions, but optimal scheduling remains an open problem [57, 58]. Balancing the trade-off between inference-time compute and accuracy is crucial for practical deployment.

Generalization and Domain Adaptation. Many planning and reflection strategies are highly task-specific. Methods trained on mathematical reasoning often fail to generalize to commonsense

or creative tasks due to domain-specific heuristics [59, 60]. Additionally, models struggle with obfuscated domains where semantic priors are removed, indicating a reliance on surface-level patterns rather than deep structural understanding [61, 54]. Developing domain-agnostic planning frameworks that can abstract high-level strategies and transfer them across diverse contexts is a key direction for future work [62, 63].

Safety and Alignment Risks. The autonomy granted by planning and self-reflection introduces new safety risks. Agents capable of modifying their own plans or executing code may inadvertently pursue misaligned goals or exploit loopholes in reward functions [64]. The "Mirror Loop" phenomenon suggests that ungrounded recursive self-critique can lead to epistemic stasis, where models converge on incorrect beliefs without external grounding [65]. Ensuring that reflective processes remain aligned with human values and do not amplify biases or hallucinations requires robust oversight mechanisms and perhaps "constitutional" constraints on agent behavior [62, 66].

Evaluation Benchmarks. Current benchmarks often fail to capture the nuances of long-horizon planning and deep reflection. Many evaluations rely on static datasets that do not test dynamic adaptability or robustness to environmental changes [67, 68]. There is a pressing need for dynamic, interactive benchmarks that assess not just final answer accuracy but also the quality of the planning process, the efficiency of resource usage, and the ability to recover from failures [69, 56].

4. Conclusion

The integration of planning and self-reflection represents a transformative shift in Large Language Model reasoning, enabling models to tackle complex, multi-step problems with unprecedented sophistication. From anticipatory planning frameworks that decouple strategy from execution to iterative self-correction loops that mimic human meta-cognition, these methods have significantly pushed the boundaries of what LLMs can achieve. Search-based approaches like Tree of Thoughts and MCTS provide robust mechanisms for exploring solution spaces, while representation engineering offers efficient, training-free interventions to steer model behavior.

However, the path toward fully autonomous, reliable reasoning agents is fraught with challenges. The fragility of intrinsic verification, the high computational costs of deliberative processes, and the risks of misalignment in agentic systems necessitate continued research. Future advancements will likely depend on hybrid architectures that seamlessly integrate neural generation with symbolic verification, efficient algorithms that dynamically allocate compute based on problem difficulty, and rigorous evaluation frameworks that assess reasoning quality beyond simple accuracy metrics. As the field matures, the synthesis of these diverse methodological families will be essential for realizing the full potential of LLMs as general-purpose reasoning engines.

References

[1] Aske Plaat, Annie Wong, Suzan Verberne. (2025). Multi-Step Reasoning with Large Language Models, a Survey. arXiv:2407.11511.

[2] Xavier Amatriain. (2024). Prompt Design and Engineering: Introduction and Advanced Methods. arXiv:2401.14423.

[3] Kaya Stechly, Karthik Valmeekam, Subbarao Kambhampati. (2024). On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks. arXiv:2402.08115.

[4] Houjun Liu. (2024). Plan of Thoughts: Heuristic-Guided Problem Solving with Large Language Models. arXiv:2404.19055.

[5] Liping Liu, Chunhong Zhang, Likang Wu. (2024). Instruct-of-Reflection: Enhancing Large Language Models Iterative Reflection Capabilities via Dynamic-Meta Instruction. arXiv:2503.00902.

[6] Shibo Hao, Yi Gu, Haodi Ma. (2023). Reasoning with Language Model is Planning with World Model. arXiv:2305.14992.

[7] Xinyi Wang, Lucas Caccia, Oleksiy Ostapenko. (2024). Guiding Language Model Reasoning with Planning Tokens. arXiv:2310.05707.

[8] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman. (2023). Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. arXiv:2310.04406.

[9] Jin Zhang, Flood Sung, Zhilin Yang. (2024). LEARNING TO PLAN BEFORE ANSWERING: SELFTEACHING LLMS TO LEARN ABSTRACT PLANS FOR PROBLEM SOLVING. arXiv:2505.00031.

[10] Jiaxin Wen, Jian Guan, Hongning Wang. (2024). UNLOCKING REASONING POTENTIAL IN LARGE LANGUAGE MODELS BY SCALING CODE-FORM PLANNING. arXiv:2409.12452.

[11] Runquan Gui, Zhihai Wang, Jie Wang. (2024). HyperTree Planning: Enhancing LLM Reasoning via Hierarchical Thinking. arXiv:2505.02322.

[12] Bingqian Lin, Yunshuang Nie, Khun Loun Zai. (2024). EvolveNav: Self-Improving Embodied Reasoning for LLM-Based Vision-Language Navigation. arXiv:2506.01551.

[13] Yuwei Yin, EunJeong Hwang, Giuseppe Carenini. (2025). SWI: Speaking with Intent in Large Language Models. arXiv:2503.21544.

[14] Kamel Alrashedy, Vriksha Srihari, Zulfiqar Zaidi. (2025). Constraints-of-Thought: A Framework for Constrained Reasoning in Language-Model-Guided Search. arXiv:2510.08992.

[15] Allen Z. Ren, Brian Ichter, Anirudha Majumdar. (2024). THINKING FORWARD AND BACKWARD: EFFECTIVE BACKWARD PLANNING WITH LARGE LANGUAGE MODELS. arXiv:2411.01790.

[16] André de Souza Loureiro, Jorge Valverde-Rebaza, Julieta Noguez. (2025). Advancing Multi-Step Mathematical Reasoning in Large Language Models Through Multi-Layered Self-Reflection with Auto-Prompting. arXiv:2506.23888.

[17] Mariana Costa, Alberlucia Rafael Soarez, Daniel Kim. (2024). Enhancing Self-Correction in Large Language Models through Multi-Perspective Reflection. arXiv:2601.07780.

[18] Sara Abdali, Can Goksen, Michael Solodko. (2024). Self-reflecting Large Language Models: A Hegelian Dialectical Approach. arXiv:2501.14917.

[19] Qinglin Zeng, Jing Yang, Keze Wang. (2025). Reflective Confidence: Correcting Reasoning Flaws via Online Self-Correction. arXiv:2512.18605.

[20] Jian Mu, Qixin Zhang, Zhiyong Wang. (2025). Self-Reflective Generation at Test Time. arXiv:2510.02919.

[21] David X. Wu, Shreyas Kapur, Anant Sahai. (2025). Synthetic Error Injection Fails to Elicit Self-Correction In Language Models. arXiv:2512.02389.

[22] Shunyu Yao, Dian Yu, Jeffrey Zhao. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601.

[23] Leon Hamm, Zlatan Ajanovic. (2025). Novelty-based Tree-of-Thought Search for LLM Reasoning and Planning. arXiv:2605.06040.

[24] Zhen-Yu Zhang, Siwei Han, Huaxiu Yao. (2024). Generating Chain-of-Thoughts with a Pairwise-Comparison Approach to Searching for the Most Promising Intermediate Thought. arXiv:2402.06918.

[25] Hui Wang, Xufeng Zhang, Chaoxu Mu. (2024). Planning of Heuristics: Strategic Planning on Large Language Models with Monte Carlo Tree Search for Automating Heuristic Optimization. arXiv:2502.11422.

[26] Chao-Chi Chen, Chin-Yuan Yeh, Hsi-Wen Chen. (2024). Self-guided Knowledgeable Network of Thoughts: Amplifying Reasoning with Large Language Models. arXiv:2412.16533.

[27] Siheng Xiong, Ali Payani, Yuan Yang. (2024). Deliberate Reasoning in Language Models as Structure-Aware Planning with an Accurate World Model. arXiv:2410.03136.

[28] Ge Yan, Chung-En Sun, Tsui-Wei (Lily) Weng. (2025). ReflCtrl: Controlling LLM Reflection via Representation Engineering. arXiv:2512.13979.

[29] Xudong Zhu, Jiachen Jiang, Mohammad Mahdi Khalili. (2025). From Emergence to Control: Probing and Modulating Self-Reflection in Language Models. arXiv:2506.12217.

[30] Ian Li, Kapilesh Guruprasad, Raunak Sengupta. (2024). Manifold-Guided Attention Steering. arXiv:2605.21770.

[31] Manan Gupta, Dhruv Kumar. (2025). Latent Phase-Shift Rollback: Inference-Time Error Correction via Residual Stream Monitoring and KV-Cache Steering. arXiv:2604.18567.

[32] Hongxiang Zhang, Yuan Tian, Tianyi Zhang. (2025). SELF-ANCHOR: LARGE LANGUAGE MODEL REASONING VIA STEP-BY-STEP ATTENTION ALIGNMENT. arXiv:2510.03223.

[33] Paul C. Bogdan, Uzay Macar, Neel Nanda. (2024). Thought Anchors: Which LLM Reasoning Steps Matter?. arXiv:2506.19143.

[34] Akbar Anbar Jafari, Gholamreza Anbarjafari. (2025). CLOSED-LOOP TRANSFORMERS: AUTOREGRESSIVE MODELING AS ITERATIVE LATENT EQUILIBRIUM. arXiv:2511.21882.

[35] Jacob Fein-Ashley, Paria Rashidinejad. (2026). Solve the Loop: Attractor Models for Language and Reasoning. arXiv:2605.12466.

[36] Ziqi Zhao, Xinyu Ma, Liu Yang. (2025). ROSD: Reflective On-Policy Self-Distillation for Language Model Reasoning across Domains. arXiv:2605.28014.

[37] Viktor Moskvoretskii, Chris Biemann, Irina Nikishina. (2025). Self-Taught Self-Correction for Small Language Models. arXiv:2503.08681.

[38] Difan Jiao, Qianfeng Wen, Blair Yang. (2025). ThinkTwice: Jointly Optimizing Large Language Models for Reasoning and Self-Refinement. arXiv:2604.01591.

[39] Lianrui Li, Dakuan Lu, Jiawei Shao. (2025). ScRPO: From Errors to Insights. arXiv:2511.06065.

[40] Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh. (2025). Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. arXiv:2503.01307.

[41] Zhuo Wang, Zhuo Zhang, Yafu Li. (2025). COTEVOL: Self-Evolving Chain-of-Thoughts for Data Synthesis in Mathematical Reasoning. arXiv:2604.14768.

[42] Mihir Parmar, Palash Goyal, Xin Liu. (2025). PLAN-TUNING: Post-Training Language Models to Learn Step-by-Step Planning for Complex Problem Solving. arXiv:2507.07495.

[43] Zhongwei Wan, Zhihao Dou, Che Liu. (2025). SRPO: Enhancing Multimodal LLM Reasoning via Reflection-Aware Reinforcement Learning. arXiv:2506.01713.

[44] Zongxian Yang, Jiayu Qian, Zegao Peng. (2025). Med-REFL: Medical Reasoning Enhancement via Self-Corrected Fine-grained Reflection. arXiv:2506.13793.

[45] Bhargavi Paranjape, Scott Lundberg, Sameer Singh. (2023). ART: Automatic multi-step reasoning and tool-use for large language models. arXiv:2303.09014.

[46] Yifan Li, Ziyi Ni, Daxiang Dong. (2024). Tree-of-Code: A Hybrid Approach for Robust Complex Task Planning and Execution. arXiv:2412.14212.

[47] Zhiyuan Ma, Jiayu Liu, Xianzhen Luo. (2025). Advancing Tool-Augmented Large Language Models via Meta-Verification and Reflection Learning. arXiv:2506.04625.

[48] Chengbo He, Bochao Zou, Xin Li. (2024). Enhancing LLM Reasoning with Multi-Path Collaborative Reactive and Reflection agents. arXiv:2501.00430.

[49] Wentao Zhang, Qunbo Wang, Zhao BoXuan. (2025). Reflecting with Two Voices: A Co-Adaptive Dual-Strategy Framework for LLM-Based Agent Decision Making. arXiv:2512.08366.

[50] Yuliang Li, Nitin Kamra, Ruta Desai. (2023). Human-Centered Planning. arXiv:2311.04403.

[51] Himabindu Thogaru, Saisubramaniam Gopalakrishnan, Zishan Ahmad. (2025). Intelligent Human-Machine Partnership for Manufacturing: Enhancing Warehouse Planning through Simulation-Driven Knowledge Graphs and LLM Collaboration. arXiv:2512.18265.

[52] Karthik Valmeekam, Matthew Marquez, Subbarao Kambhampati. (2024). Can Large Language Models Really Improve by Self-critiquing Their Own Plans?. arXiv:2310.08118.

[53] Lixing Lin, Juli Youin, Yue Li. (2024). Reflect-Guard: Enhancing LLM Safeguards against Adversarial Prompts via Logical Self-Reflection. arXiv:2605.24834.

[54] Augusto B. Corrêa, André G. Pereira, Jendrik Seipp. (2025). The 2025 Planning Performance of Frontier Large Language Models. arXiv:2511.09378.

[55] Xingyu Chen, Jiahao Xu, Tian Liang. (2025). Do NOT Think That Much for 2 + 3 = ? On the Overthinking of o1-Like LLMs. arXiv:2412.21187.

[56] Zhiyuan Li, Yi Chang, Yuan Wu. (2025). THINK-Bench: Evaluating Thinking Efficiency and Chain-of-Thought Quality of Large Reasoning Models. arXiv:2505.22113.

[57] Tianyu Fu, Yichen Vou, Zekai Chen. (2025). THINK-AT-HARD: SELECTIVE LATENT ITERATIONS TO IMPROVE REASONING LANGUAGE MODELS. arXiv:2511.08577.

[58] Juming Xiong, Kevin Guo, Congning Ni. (2025). Learning When to Sample: Confidence-Aware Self-Consistency for Efficient LLM Chain-of-Thought Reasoning. arXiv:2603.08999.

[59] Zongqian Wu, Baoduo Xu, Ruochen Cui. (2024). Rethinking Chain-of-Thought from the Perspective of Self-Training. arXiv:2412.10827.

[60] Sachith Gunasekara, Yasiru Ratnayake. (2025). Effects of structure on reasoning in instance-level SELF-DISCOVER. arXiv:2507.03347.

[61] Karthik Valmeekam, Kaya Stechly, Subbarao Kambhampati. (2024). LLMs Still Can't Plan; Can LRMs? A Preliminary Evaluation of OpenAI's o1 on PlanBench. arXiv:2409.13373.

[62] Manasa Bharadwaj, Nikhil Verma, Kevin Ferreira. (2025). OmniReflect: Discovering Transferable Constitutions for LLM agents via Neuro-Symbolic Reflections. arXiv:2506.17449.

[63] Augusto B. Corrêa, Yoav Gelberg, Luckeciano C. Melo. (2024). Iterative Deployment Improves Planning Skills in LLMs. arXiv:2512.24940.

[64] Aske Plaat, Max van Duijn, Niki van Stein. (2025). Agentic Large Language Models, a survey. arXiv:2503.23037.

[65] Bentley DeVilling. (2025). The Mirror Loop: Recursive Non-Convergence in Generative Reasoning Systems. arXiv:2510.21861.

[66] Zhengwei Tao, Ting-En Lin, Xiancai Chen. (2024). A Survey on Self-Evolution of Large Language Models. arXiv:2404.14387.

[67] Jianghao Chen, Zhenlin Wei, Zhenjiang Ren. (2025). LR2Bench: Evaluating Long-chain Reflective Reasoning Capabilities of Large Language Models via Constraint Satisfaction Problems. arXiv:2502.17848.

[68] Xueyang Zhou, Guiyao Tie, Guowen Zhang. (2025). Exploring the Necessity of Reasoning in LLM-based Agent Scenarios. arXiv:2503.11074.

[69] Shubham Parashar, Blake Olson, Sambhav Khurana. (2025). Inference-Time Computations for LLM Reasoning and Planning: A Benchmark and Insights. arXiv:2502.12521.